Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Inteligencia Artificial • IC-6200

Proyecto Semestral: Agente de IA Aplicada

Registro del desarrollo de las Etapas 1, 2 y 3

Bitácora de Trabajo

Decisiones, bloqueos, acuerdos e hitos del equipo

Bitácora • Decisiones técnicas • Prevención de fuga • Trazabilidad

PROFESOR

Kenneth Roberto Obando Rodríguez

EQUIPO

Adriel S. Chaves Salazar	2021031465
Daniel Duarte Cordero	2022012866
Sebastián Hernández Bonilla	2022093651

II Semestre, 2026 • Cartago, Costa Rica

Índice

1 Propósito y convenciones **1**

1.1 Propósito . . . 1
1.2 El proyecto en un párrafo . . . 1
1.3 Equipo y responsabilidades . . . 1
1.4 Cómo se leen las entradas . . . 2
1.5 Cronología general . . . 2
1.6 Mapa de la evidencia . . . 3
1.7 Reglas que se respetaron durante todo el proyecto . . . 4

2 Etapa 1: estado del arte y diseño **4**

2.1 Resumen de ramas . . . 5
2.2 Bitácora . . . 5
2.3 Decisiones consolidadas . . . 10
2.4 Retrospectiva . . . 11

3 Etapa 2: modelado, explicabilidad y evaluación **11**

3.1 Resumen de ramas . . . 11
3.2 Cómo se construyó el sistema, parte por parte . . . 11
3.3 Rama 03: datos y entorno . . . 12
3.4 Rama 04: entrenamiento de la escalera de modelos . . . 14
3.5 Rama 05: evaluación final . . . 21
3.6 Rama 06: artículo y cierre . . . 24
3.7 Cómo protegimos los datos . . . 24
3.8 Cómo aseguramos que todo funcionara . . . 25
3.9 Registro de experimentos . . . 25
3.10 Decisiones consolidadas . . . 27
3.11 Retrospectiva . . . 28

4 Etapa 3: agente, validación, ética y artículo final **28**

4.1 Resumen de ramas . . . 28
4.2 Lo que recibió de la Etapa 2 . . . 29
4.3 Cómo se construyó, parte por parte . . . 29
4.4 Rama 07: integración del agente . . . 29
4.5 Rama 08: validación y simulador . . . 30
4.6 Rama 09: ética, artículo y documentación . . . 33
4.7 Cómo aseguramos que el agente fuera confiable . . . 33
4.8 Retrospectiva . . . 34
4.9 Trabajo pendiente heredado . . . 34

A Reuniones y sesiones de trabajo **34**
A.1 Índice 34
A.2 Actas 35

Índice de figuras

2.1 Primera reunión del equipo (21/08/2026), revisando conjuntos de datos en Kaggle. 5
2.2 Sesión de revisión en la que se confirmó el tema (26/08/2026). 6
2.3 Sesión de trabajo de la Etapa 1 (30/08/2026). 7
2.4 Revisión final del artículo de la Etapa 1 (06/09/2026). 9
3.1 Orden de construcción de la Etapa 2. 12
3.2 Dos resultados de la evaluación final, tomados del cuaderno 06. 23
4.1 Orden de construcción de la Etapa 3. Cada pieza se probó antes de empezar la siguiente. 29

Índice de tablas

1.1 Integrantes y responsabilidades principales. 1
1.2 Ramas del proyecto, en orden cronológico. 3
1.3 Evidencia de la Etapa 1 y de los baselines B2 y B3. 3
1.4 Evidencia de B4, de los experimentos y de la evaluación final. 4
2.1 Ramas de la Etapa 1 y su resultado. 5
2.2 Decisiones técnicas de la Etapa 1. 10
2.3 Retrospectiva de la Etapa 1. 11
3.1 Ramas de la Etapa 2 y su resultado. 11
3.2 B2: finalistas en validación (`25-10-26_B2SeleccionFinalC.md`). 17
3.3 B3: finalistas con los mismos hiperparámetros en los tres escenarios (AUC en validación). 19
3.4 AUC en validación de la escalera de modelos al cerrar la rama 04. 21
3.5 AUC de B4 en prueba (intervalo del 95 %) y veredicto de las hipótesis. . 23
3.6 Vías de fuga y cómo se cerró cada una. 25
3.7 Corridas de la escalera de modelos (B0 a B4). 26
3.8 Corridas de experimentos y evaluación final. 26
3.9 Decisiones de la Etapa 2: datos y baselines (11/10 a 25/10). 27
3.10 Decisiones de la Etapa 2: B4, experimentos y evaluación final (26/10 a 06/11). 27
3.11 Retrospectiva de la Etapa 2. 28
4.1 Ramas de la Etapa 3 y su resultado. 28
4.2 Piezas que la Etapa 2 dejó listas para el agente. 29
4.3 Retrospectiva de la Etapa 3. 34
A.1 Reuniones y sesiones registradas. 35

1. Propósito y convenciones

1.1. Propósito

Esta bitácora registra cómo se construyó el proyecto: qué se decidió, por qué, qué bloqueó el avance y dónde está la evidencia de cada afirmación. No es el artículo ni el plan de desarrollo. El artículo reporta resultados; el plan (`PlanDeDesarrollo.md`) declara lo que se quería hacer; esta bitácora cuenta lo que realmente pasó.

El énfasis está en el razonamiento humano detrás de cada paso: cómo se eligieron los modelos de referencia, cómo se diseñó la lógica de entrenamiento, cómo se protegieron los datos contra la fuga de información y cómo se comprobó en cada momento que todo funcionara. Esa es la parte del trabajo que no puede reconstruirse leyendo el historial del repositorio.

1.2. El proyecto en un párrafo

El proyecto estima la probabilidad de que gane el jugador que actúa en una partida del Pokémon Trading Card Game, a partir de un único estado observable del tablero. Usa aprendizaje automático clásico sobre datos tabulares (Track A) y un corpus público de partidas entre agentes automáticos. La dificultad principal no es el modelo sino la evaluación: cada partida produce más de un centenar de estados casi idénticos con la misma etiqueta, así que repartirlos al azar entre entrenamiento y prueba inflaría cualquier métrica. Por eso toda partición, validación e intervalo se agrupa por partida.

1.3. Equipo y responsabilidades

Tabla 1.1: Integrantes y responsabilidades principales.

Integrante	**Eje (Etapa 1)**	**Responsabilidad principal**
Adriel S. Chaves Salazar	Eje B: técnicas del *track*	Etapa 1: revisión del eje B y ensamblaje del artículo. Etapa 2: preparación de datos, entorno de Colab, almacén en Kaggle, infraestructura, revisión de la lógica de B4, experimentos (LightGBM, ablaciones, H2), ejecución del cuaderno de evaluación final y redacción del artículo.
Daniel Duarte Cordero	Eje A: problema y dominio	Etapa 1: revisión del eje A y metodología del artículo. Etapa 2: implementación y entrenamiento de B0 a B4, búsquedas de hiperparámetros, auditorías de los baselines y protocolo de LightGBM.
Sebastián Hernández Bonilla	Eje C: conjunto de datos	Etapa 1: revisión del eje C, auditoría del corpus y tabla comparativa. Etapa 2: diagnóstico del proyecto, protocolo de evaluación final, cuaderno 06 y documentación de resultados.

Repositorio: `Applied-AI-Agent-Project`. Las reglas de ramas y commits están en `docx/00ReglasGit/ReglasGit.md`; el plan, en `docx/01SeguimientoDeTareas/PlanDeDesarrollo.md`; la declaración de uso de IA, en `docx/03Evidencia/01UsoDeIA/`.

1.4. Cómo se leen las entradas

Cada entrada abre con un encabezado que indica la fecha, el título, los responsables y la rama donde ocurrió el trabajo. Dentro de la entrada, cuatro recuadros concentran lo importante:

Decisión. La opción que se eligió.
Por qué. La razón, con su evidencia.
Descartado. Las alternativas consideradas y por qué se dejaron.

Bloqueo. Lo que detuvo el avance.
Impacto. Qué retrasó y cuánto.
Solución. Cómo se resolvió.

Hito. Lo que se logró.
Evidencia. Dónde está la prueba: commit, archivo, figura o captura.

Uso de IA: herramienta: la herramienta • para qué: para qué se usó • verificado: qué revisó el equipo a mano

1.5. Cronología general

La Tabla 1.2 resume las ramas de trabajo en orden. Las fechas salen del historial de commits del repositorio.

Tabla 1.2: Ramas del proyecto, en orden cronológico.

Rama	Fechas	Responsables	Resultado
Etapa 1			
`TrackConfirmation`	21/08	Equipo	Repositorio creado (PR #1)
`00TrackConfirmation`	21/08–30/08	Equipo	Tema y *track* confirmados; auditoría del corpus (PR #2)
`01LiteratureReview`	30/08–05/09	Cada uno su eje	Revisión de 21 artículos en tres ejes (PR #3)
`02PaperAndDelivery`	05/09–17/09	Equipo	Artículo preliminar entregado (PR #4, #6, #7)
Etapa 2			
`03DataCollectionAndEnvironmentSetup`	04/10–11/10	Adriel	Tabla de estados, particiones y pruebas de fuga (PR #8)
`04MainAIModelTrainingAndDevelopment`	11/10–30/10	Daniel y Adriel	B0 a B4, experimentos y modelo final (PR #9)
`05FinalModelChecks`	30/10–05/11	Sebastián y Adriel	Evaluación única en prueba e hipótesis (PR #10)
`06PendientesDelArticulo`	05/11–06/11	Adriel	Artículo con resultados, discusión y conclusiones

1.6. Mapa de la evidencia

Cada afirmación de esta bitácora remite a un archivo de `docx/03Evidencia/`. Las Tablas 1.3 y 1.4 los ordenan por fecha.

Tabla 1.3: Evidencia de la Etapa 1 y de los baselines B2 y B3.

Archivo	Qué registra
`21-8-26_PrimeraReunionDelEquipo.png`	Primera reunión del equipo y exploración de conjuntos de datos en Kaggle
`26-8-26_ConfirmacionDelTema.png`	Sesión de revisión en la que se confirmó el tema
`30-8-26_DesarrolloDeLaEtapa1.png`	Sesión de trabajo de la Etapa 1
`06-9-26_RevisionFinalEtapa1.png`	Revisión final del artículo antes de la entrega
`12-10-26_AcuerdosTecnicosModelado.md`	Reglas de modelado acordadas antes de entrenar: B1, validación cruzada, pesos por partida, búsquedas
`18-10-26_B2MemoriaAcotada.md`	Cambio de B2 a entrenamiento por lotes para caber en la RAM de Colab
`19-10-26_B2ResultadosYFinalistas.md`	Primera búsqueda de regularización de B2 y lectura de la falta de convergencia
`19-10-26_B3B4Checkpoints.md`	Puntos de control para reanudar B3 y B4 si Colab se corta
`23-10-26_AuditoriaBaselinesYCoste.md`	Auditoría del costo de las búsquedas y de los errores de reanudación
`25-10-26_B2SeleccionFinalC.md`	Elección final de $C = 0.1$ para B2

Tabla 1.4: Evidencia de B4, de los experimentos y de la evaluación final.

Archivo	Qué registra
`05_entrenamiento_B4_independiente_original.ipynb`	Cuaderno original de B4 con búsqueda única y tope de 4000 árboles
`28-10-26_ProtocoloLightGBM.md`	Presupuesto y regla de comparación con LightGBM, fijados antes de ejecutar
`28-10-26_SeleccionB3yDefinicionH2.md`	Regla de macro-AUC para B3 y definición operativa de H2
`28-10-26_VerificacionFinalistasB2B3.json`	Hashes y metadatos de los modelos finales de B2 y B3
`29-10-26_AuditoriaBaselinesB0B3.md`	Cumplimiento de B0 a B3 frente al contrato de modelado
`02-11-26_DiagnosticoFinalModelChecks.md`	Estado del proyecto y riesgos al iniciar la evaluación final
`02-11-26_ProtocoloEvaluacionFinal.md`	Reglas de la evaluación final, congeladas antes de abrir prueba
`04-11-26_ResultadosEvaluacionFinal.md`	Resultados en prueba, calibración, SHAP e hipótesis
`01UsoDeIA/`	Declaración de uso de IA de las Etapas 1 y 2

1.7. Reglas que se respetaron durante todo el proyecto

- **La partida es la unidad independiente.** Ninguna partición, pliegue o intervalo separa estados de una misma partida.
- **El conjunto de prueba se usa una sola vez.** Ninguna decisión depende de sus números.
- **Ningún resultado se genera con IA.** Cada cifra viene de una celda ejecutada por el equipo y queda registrada con su SHA-256 y su commit.
- **Las credenciales nunca se escriben** en cuadernos, archivos versionados ni conversaciones.
- **Una decisión se documenta antes de ejecutar**, no después de ver el resultado.

2. Etapa 1: estado del arte y diseño

Objetivo de la etapa. Elegir el tema, revisar la literatura en tres ejes, auditar el corpus y entregar un artículo IEEE preliminar con la pregunta de investigación, las hipótesis y el protocolo experimental. Entrega: 6 de septiembre de 2026. Peso: 30 %.

2.1. Resumen de ramas

Tabla 2.1: Ramas de la Etapa 1 y su resultado.

Rama	Fechas	Responsables	Resultado
`TrackConfirmation`	21/08	Adriel	Repositorio, licencia y reglas de Git
`00TrackConfirmation`	21/08–30/08	Equipo	Track A y tema aprobados; corpus auditado
`01LiteratureReview`	30/08–05/09	Daniel (A), Adriel (B), Sebastián (C)	Revisión por ejes y bibliografía verificada
`02PaperAndDelivery`	05/09–17/09	Equipo	Artículo preliminar, declaración de IA y entrega

2.2. Bitácora

2026-08-21 • Primera reunión y creación del repositorio

Responsables: Adriel, Daniel, Sebastián • Rama: `TrackConfirmation`

El equipo se reunió por primera vez por llamada (Fig. 2.1). Se revisaron conjuntos de datos públicos en Kaggle, entre ellos las repeticiones del PTCG AI Battle Challenge publicadas por The Pokémon Company. Ese mismo día Adriel creó el repositorio con su licencia, su `.gitignore` y las reglas de ramas y commits.

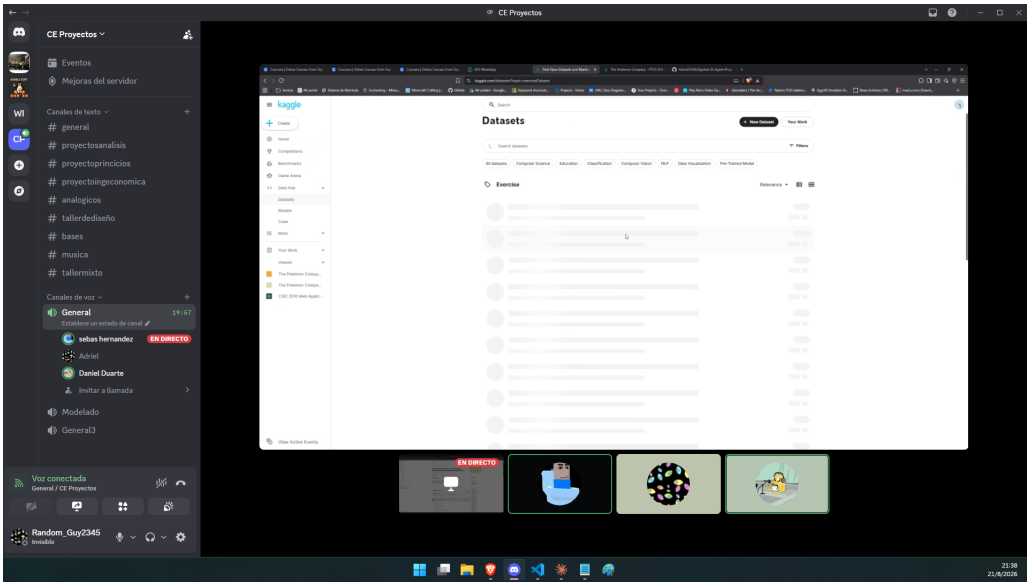


Figura 2.1: Primera reunión del equipo (21/08/2026), revisando conjuntos de datos en Kaggle.

Decisión. Organizar el repositorio en tres niveles de ramas (`master`, `Stage<N>` y ramas de trabajo numeradas) y usar Conventional Commits.

Por qué. La autoría individual se califica por el historial de commits, y una rama por bloque de trabajo hace visible quién hizo qué y cuándo.

Descartado. Ramas personales y una rama `develop`: mezclaban trabajos y ocultaban la etapa a la que pertenecía cada cambio.

Hito. Repositorio operativo.

Evidencia. PR #1 y commit "docs(repo): add branching and commit conventions".

2026-08-26 • Confirmación del tema

Responsables: Adriel, Sebastián • Rama: `00TrackConfirmation`

En una sesión de revisión (Fig. 2.2) se cerró el tema: estimar la probabilidad de victoria en el Pokémon TCG desde el estado de la partida, como clasificación tabular.

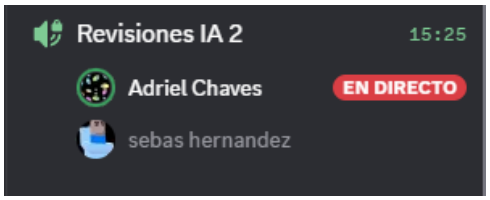


Figura 2.2: Sesión de revisión en la que se confirmó el tema (26/08/2026).

Decisión. Track A (aprendizaje clásico sobre datos tabulares) con el corpus del PTCG AI Battle Challenge.

Por qué. El corpus es público, grande y tiene etiqueta clara (quién gana). El problema de evaluación que plantea, miles de estados correlacionados por partida, es justo el que el Track A pide tratar con rigor: fuga, baselines y protocolo.

Descartado. Una primera dirección de aprendizaje por refuerzo (planificación de rutas de cobertura en terreno desconocido). Se abandonó tras la reevaluación del equipo y la retroalimentación del profesor.

Uso de IA: herramienta: Claude • para qué: explorar direcciones del proyecto y probar si la pregunta era falsable • verificado: la elección del tema, del *track* y la decisión de dejar la dirección de refuerzo fueron del equipo

2026-08-30 • Auditoría del corpus y base de investigación

Responsables: Adriel, Daniel, Sebastián • Rama: `00TrackConfirmation`

En una sesión de trabajo de los tres (Fig. 2.3) se cerró la confirmación del *track* y se fijó la base de investigación. La auditoría del corpus analizó diez días entre el 17 de junio y el 22 de julio: 52 085 partidas y 7 038 378 observaciones, unos 180 GiB, sin errores

de lectura. También midió que, en cada paso, la vista del jugador que espera es 98.8 % idéntica a su vista anterior.

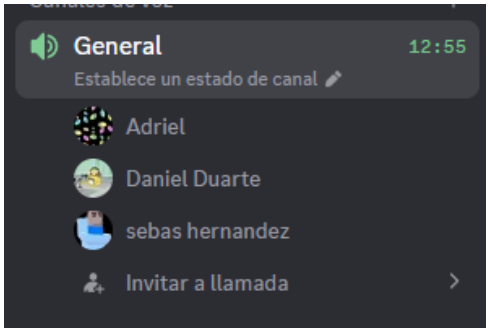


Figura 2.3: Sesión de trabajo de la Etapa 1 (30/08/2026).

Decisión. Agrupar por partida toda partición, validación cruzada e intervalo.

Por qué. La auditoría midió que una partición aleatoria por fila colocaría prácticamente todas las partidas a ambos lados de la división: el modelo se evaluaría sobre estados casi idénticos a los que vio. La decisión se apoya en una medición sobre el propio corpus, no en una analogía.

Descartado. Partición aleatoria por fila. Solo se conserva como demostración del sesgo, nunca para reportar resultados.

Decisión. Tomar solo la vista del jugador que debe actuar y describir cada estado desde su perspectiva (propio, oponente y diferencia).

Por qué. La vista del jugador que espera repite casi siempre la anterior y duplicaría filas sin información nueva. La perspectiva propio/oponente evita que el modelo aprenda "la posición 0 gana más".

Descartado. Usar ambas vistas por paso, o columnas por posición fija en la mesa.

Decisión. Declarar una lista de campos prohibidos: mano del oponente, contenido de los premios, resultado, recompensas, nombres de agentes e historial de jugadas.

Por qué. Son información que el jugador no ve en ese instante o que revela el desenlace. Usarlos sería fuga.

Descartado. Confiar en que el modelo "no los usaría": la fuga se previene por construcción, no por inspección posterior.

Hito. Tema confirmado y corpus auditado.

Evidencia. PR #2 y commit "docs(repo): close track confirmation and establish the Stage 1 research base".

2026-08-30 a 2026-09-05 • Revisión de literatura por ejes

Responsables: Daniel (eje A), Adriel (eje B), Sebastián (eje C) • Rama: `01LiteratureReview`

Cada integrante revisó un eje: el problema y su dominio (A), las técnicas del *track* (B) y el conjunto de datos (C). Se seleccionaron 21 artículos arbitrados. Cada uno resolvió los DOI de sus fuentes y leyó el texto completo antes de agregarlas a la bibliografía.

Decisión. Una entrada solo entra al `.bib` principal si su DOI resuelve y su contenido corresponde a lo que se le atribuye.

Por qué. Las herramientas de IA proponen metadatos plausibles pero a veces falsos; una cita incorrecta invalida el argumento que sostiene.

Descartado. Aceptar los metadatos que sugiere una herramienta. Cuando un dato no pudo confirmarse, se dejó vacío en lugar de rellenarlo con una suposición.

Decisión. Usar como baseline de literatura el AUC de 0.78 a 0.80 del desafío AAIA'17 sobre Hearthstone.

Por qué. Es el antecedente más cercano: un juego de cartas por turnos con predicción de victoria desde el estado.

Descartado. Usar cifras de juegos de estrategia en tiempo real, cuyo estado y dinámica son muy distintos.

Bloqueo. Varios candidatos eran *preprints* sin arbitraje.

Impacto. Retrasó el cierre de la bibliografía.

Solución. Se marcaron como tales y se excluyeron, o se consultó con el profesor antes de incluirlos.

Uso de IA: herramienta: Consensus, Claude y ChatGPT • para qué: buscar candidatos, clasificarlos y detectar *preprints* • verificado: cada DOI se resolvió a mano y cada artículo se leyó

Hito. Revisión de literatura integrada al borrador.

Evidencia. PR #3.

2026-09-05 a 2026-09-06 • Ensamblaje del artículo y revisión final

Responsables: Adriel, Daniel, Sebastián • Rama: `02PaperAndDelivery`

Se integraron la introducción, los trabajos relacionados, la tabla comparativa, el baseline de literatura, la metodología, el análisis técnico y las consideraciones éticas. La revisión final duró más de hora y media con los tres integrantes (Fig. 2.4).

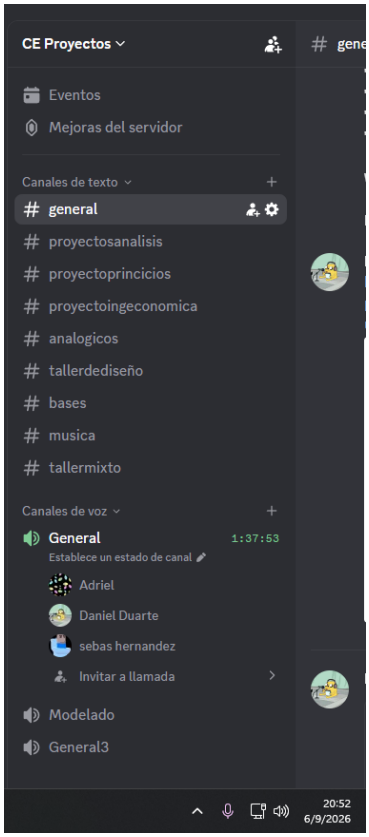


Figura 2.4: Revisión final del artículo de la Etapa 1 (06/09/2026).

Decisión. Formular tres hipótesis refutables (H1 a H3) sin umbrales numéricos fijos.

Por qué. Sin resultados previos sobre este corpus, cualquier umbral sería arbitrario. Cada hipótesis declara qué resultado la refutaría.

Descartado. Hipótesis del tipo “el modelo funciona bien”, que no pueden fallar.

Decisión. Una escalera de cinco modelos, B0 a B4, en la que cada nivel responde una pregunta.

Por qué. B0 (probabilidad a priori) fija el piso; B1 (heurística) pregunta si hace falta aprendizaje; B2 (regresión logística) si la relación es lineal; B3 (bosque) si hay interacciones; B4 (potenciación del gradiente) cuánto añade la complejidad.

Descartado. Comparar solo contra un modelo trivial, que no dice si la complejidad vale la pena.

Decisión. Tres escenarios de evaluación: partidas nuevas (A), días posteriores (B) y mazos reservados (C).

Por qué. Cada uno responde una pregunta de generalización distinta, y se reportan por separado.

Descartado. Un solo conjunto de prueba aleatorio, que no mide el cambio de la población ni la memorización de mazos.

Hito. Artículo preliminar entregado.

Evidencia. PR #4, PR #5 (`Stage1` → `master`) y etiqueta `v1.0.0`.

2026-09-06 a 2026-09-17 • Correcciones posteriores a la entrega

Responsables: Adriel • Rama: `02PaperAndDelivery`

Se corrigieron figuras, la integración de las referencias y el título de los autores.

Hito. Correcciones integradas.

Evidencia. PR #6 (etiqueta `v1.0.1`) y PR #7.

2.3. Decisiones consolidadas

Tabla 2.2: Decisiones técnicas de la Etapa 1.

Fecha	Decisión	Por qué	Sección del artículo
26/08	Track A con el corpus del PTCG AI Battle Challenge	Datos públicos con etiqueta clara y un reto de evaluación real	Introducción
30/08	Agrupar todo por partida	Medido: la partición por fila mezcla casi todas las partidas	Metodología, análisis técnico
30/08	Una fila por estado del jugador que actúa, en perspectiva propia	Evita duplicados y el sesgo de posición	Metodología
30/08	Lista de campos prohibidos	Previene la fuga por construcción	Análisis técnico
05/09	Baseline de literatura: AUC 0.78–0.80 (Hearthstone)	Antecedente más cercano	Baseline de literatura
06/09	Hipótesis H1 a H3 refutables	Sin umbrales arbitrarios	Introducción
06/09	Escalera B0–B4 y escenarios A, B, C	Cada nivel y escenario responde una pregunta	Metodología

2.4. Retrospectiva

Tabla 2.3: Retrospectiva de la Etapa 1.

Pregunta	Respuesta
¿Qué funcionó?	Repartir la literatura por ejes y medir el corpus antes de diseñar el protocolo. La decisión de agrupar por partida quedó respaldada por una medición propia.
¿Qué nos bloqueó?	Verificar los metadatos de cada referencia contra la fuente primaria tomó más tiempo del previsto.
¿Qué estimamos mal?	El tiempo de ensamblaje del artículo: la mayor parte del trabajo se concentró en los últimos dos días.
¿Qué cambia para la Etapa 2?	Traducir la documentación al español, reemplazar el plan ágil por un plan de desarrollo por ramas y abrir una rama propia para los datos antes de entrenar.

3. Etapa 2: modelado, explicabilidad y evaluación

Objetivo de la etapa. Ejecutar el diseño de la Etapa 1: construir la tabla de estados, entrenar la escalera de modelos B0 a B4 bajo un protocolo sin fuga, evaluar una sola vez en prueba, explicar el modelo final con SHAP, analizar la calibración (módulo del grupo de tres) y escribir la metodología definitiva y los resultados del artículo. Peso: 45 %.

3.1. Resumen de ramas

Tabla 3.1: Ramas de la Etapa 2 y su resultado.

Rama	Fechas	Responsables	Resultado
`03DataCollectionAndEnvironmentSetup`	04/10–11/10	Adriel	55 196 partidas en Parquet, 62 características, particiones A/B/C y 24 pruebas de fuga
`04MainAIModelTrainingAndDevelopment`	11/10–30/10	Daniel y Adriel	B0 a B4 entrenados, LightGBM, ablaciones, H2 y modelo final exportado
`05FinalModelChecks`	30/10–05/11	Sebastián y Adriel	Protocolo congelado, evaluación única en prueba, calibración, SHAP y veredicto de H1 a H3
`06PendientesDelArticulo`	05/11–06/11	Adriel	Artículo corregido, con resultados, discusión y conclusiones

3.2. Cómo se construyó el sistema, parte por parte

La Fig. 3.1 muestra el orden en que se construyó cada pieza. Cada bloque se cerró y se verificó antes de empezar el siguiente: no se entrenó ningún modelo hasta tener las

particiones probadas, y no se abrió el conjunto de prueba hasta tener todos los modelos, la calibración y la revisión de fuga listos.

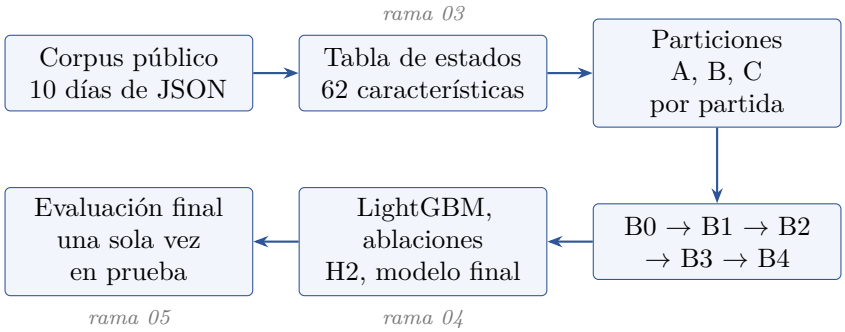


Figura 3.1: Orden de construcción de la Etapa 2.

3.3. Rama 03: datos y entorno

2026-10-04 • Apertura de la etapa y lectura de una partida real

Responsables: Adriel • Rama: `03DataCollectionAndEnvironmentSetup`

Se tradujo la documentación al español, se reemplazó el plan ágil de la Etapa 1 por `PlanDeDesarrollo.md` y se preparó el entorno de Colab. Antes de escribir el extractor se inspeccionó el JSON de una partida completa para entender el formato real:

- En cada paso hay dos vistas. El jugador `ACTIVE` debe decidir; el `INACTIVE` espera y su vista va un paso atrasada.
- La mano del oponente llega como `null` y los premios como una lista de `null`: la información oculta ya viene protegida.
- Como todos los premios son nulos, los premios restantes se cuentan por el *largo* de la lista, no por los elementos no nulos.

Decisión. Escribir la lógica en `src/ptcg_wp/*.py` y dejar los cuadernos solo para ejecutar y mostrar resultados.

Por qué. El código en módulos se puede probar con `pytest`, revisar en un PR e importar desde el agente de la Etapa 3.

Descartado. Lógica dentro de las celdas: imposible de probar y de reutilizar.

Hito. Primer extractor: 61 características propio/oponente, marca de preparación y descarte de empates.

Evidencia. Commit “feat(features): extraer estados del jugador activo y añadirlos a la ingesta”.

2026-10-11 • Estados resueltos, pruebas de fuga y almacén en Kaggle

Responsables: Adriel • Rama: `03DataCollectionAndEnvironmentSetup`

Al revisar las partidas completas se encontró que algunos estados ya muestran el desenlace: un jugador sin premios por tomar o sin Pokémon en juego. Se agregaron la columna `decided` y el filtro `usable`, el número de turno público (`game_turn`) y los identificadores de los Pokémon activos. Los agentes se seudonimizaron con SHA-256 truncado.

Decisión. Excluir los estados de preparación y los ya resueltos de toda tabla de modelado.

Por qué. Un estado resuelto tiene el desenlace a la vista; incluirlo inflaría el desempeño con casos triviales. Son el 0.2 % de los estados.

Descartado. Conservarlos: el modelo aprendería a leer un tablero ya terminado.

Decisión. Probar la ausencia de fuga con pruebas automáticas que *intentan* provocarla.

Por qué. Las pruebas llenan la mano rival y los premios con cartas reales, cambian el resultado y las recompensas, y cortan los pasos futuros. En todos los casos las características deben quedar idénticas. Si alguien cambia el extractor y filtra algo oculto, la prueba falla.

Descartado. Revisar a mano el código del extractor: no protege contra cambios futuros.

Decisión. Guardar las tablas procesadas en un dataset privado de Kaggle, nunca en Git.

Por qué. Son gigabytes de datos y el repositorio debe guardar la receta, no los datos. Kaggle permite que los tres integrantes bajen exactamente las mismas tablas.

Descartado. Versionar los Parquet en Git o que cada uno reconstruyera las tablas (unos 12 minutos por día).

Bloqueo. Los compañeros recibían un error 404 al bajar el dataset completo, y la herramienta `kaggle` informaba éxito aunque la publicación fallara.

Impacto. Nadie más que Adriel podía trabajar con las tablas.

Solución. Se pasó a descargar archivo por archivo y `store.py` empezó a leer la salida de `kaggle` y fallar si no se bajaba nada.

Uso de IA: herramienta: Claude • para qué: implementar la ingesta, el extractor, las pruebas y el almacén en Kaggle, y diagnosticar los errores de Colab • verificado: Adriel ejecutó cada celda y comparó los conteos con lo esperado antes de cada commit

2026-10-11 • Particiones, cobertura de cartas y diccionario de datos

Responsables: Adriel • Rama: `03DataCollectionAndEnvironmentSetup`

Se construyeron las particiones de los tres escenarios, agrupadas por partida, con la semilla 2026.

Decisión. Escenario B por días completos: del 18 al 25 de junio a entrenamiento, el 7 de julio a validación, el 11 a calibración, y el 15 y el 22 a prueba. El 17 de junio queda fuera.

Por qué. Partir dentro de un día mezclaría el presente con el futuro. El 17 de junio fue un arranque parcial de la competencia. Ningún día mostró una concentración anómala de mazos (mínimo 274 mazos distintos frente a una mediana cercana a 450).

Descartado. Cortes por fecha y hora dentro de un día; reservar un día anómalo, que al final no existió.

Decisión. Escenario C por arquetipos: toda partida con alguno de 33 Pokémon principales reservados va a prueba (15.1 % de las partidas).

Por qué. El corpus no trae mazos completos. El Pokémon que más tiempo pasa activo identifica bien el arquetipo y permite reservar estrategias enteras.

Descartado. Reservar mazos completos: casi no se repiten, así que la reserva sería arbitraria.

Decisión. Cobertura de cartas: en A y C toda carta activa debe aparecer en entrenamiento; si no, se mueve a entrenamiento la partida más corta que la contiene. En B no se mueve nada.

Por qué. Sin esta regla, la ablación de H2 se evaluaría sobre cartas que el modelo nunca vio. En B, mover partidas futuras a entrenamiento sería fuga temporal, así que las cartas nuevas (0.86 a 1.20 % de las filas) se aceptan como parte del cambio que B mide.

Descartado. Regenerar la partición con otra semilla hasta que la cobertura saliera completa.

Al revisar la tabla se encontró que la tasa de victoria de la posición 0 subía de 0.507 a 0.559 con los días. La auditoría mostró la causa: el jugador que empieza ocupa la posición 0 en el 76.4 % de las partidas del 17 de junio y en el 100 % de las del 22 de julio. No es una ventaja de la posición sino de mover primero, y por eso el modelo recibe `first_player_is_me` y no la posición.

Hito. Tabla final verificada: 55 196 partidas, 7 540 389 estados, 7 286 386 filas utilizables, 119 empates excluidos, 0 partidas anómalas y 24 pruebas automáticas.

Evidencia. PR #8, `DiccionarioDeDatos.md` y cuaderno `01_exploracion`.

3.4. Rama 04: entrenamiento de la escalera de modelos

2026-10-12 • Acuerdos técnicos antes de entrenar

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Antes de escribir una sola línea de entrenamiento, Daniel fijó por escrito las reglas de modelado (`12-10-26_AcuerdosTecnicosModelado.md`).

Decisión. Cada partida pesa lo mismo, en el entrenamiento y en las métricas: cada estado recibe peso $1/n$, con n los estados de su partida.

Por qué. Sin esa corrección, una partida larga pesaría diez veces más que una corta solo por durar más.

Descartado. Pesar cada fila por igual.

Decisión. B1 es una heurística fija: (*premios rival* − *premios propios*) + *diferencia de PV/unidad de PV* + *diferencia de banca*. Solo la pendiente y el intercepto que la convierten en probabilidad se ajustan con entrenamiento.

Por qué. B1 debe responder si hace falta aprendizaje automático; si sus pesos se aprendieran, sería otro modelo supervisado.

Descartado. Ajustar los pesos de la heurística con las etiquetas.

Decisión. Seleccionar con validación cruzada agrupada por partida (`GroupKFold`), comparar finalistas en validación y dejar prueba cerrada.

Por qué. Cada conjunto tiene un solo uso: entrenamiento ajusta, validación elige, calibración recalibra y prueba reporta.

Descartado. `KFold` o `StratifiedKFold` sin grupos, que mezclarían estados de una partida entre pliegues.

Decisión. La parada temprana de B4 vigila un grupo de partidas completas, separado dentro del entrenamiento. Se fija `scikit-learn==1.7.2`, la primera versión que acepta datos de vigilancia externos.

Por qué. La vigilancia interna por defecto separa filas al azar y pondría estados de una misma partida a ambos lados.

Descartado. La vigilancia por defecto de scikit-learn.

Uso de IA: herramienta: Claude • para qué: discutir las alternativas de protocolo • verificado: las reglas quedaron escritas y fechadas antes de ejecutar

2026-10-12 a 2026-10-13 • B0 y B1

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Bloqueo. Cargar todas las columnas de todos los días agotaba la RAM de Colab.

Impacto. El cuaderno de B0 y B1 no terminaba.

Solución. Leer solo las columnas necesarias y recorrer las tablas día por día.

Hito. B0 y B1 medidos en validación para A, B y C. AUC de B1: 0.677, 0.668 y 0.671.
Evidencia. Cuaderno `02_baselines_B0_B1`.

2026-10-15 a 2026-10-18 • B2 con todo el entrenamiento en la RAM de Colab

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

La primera versión de B2 cargaba la matriz completa y fallaba por memoria. Una muestra de 5 000 partidas lo hacía caber, pero desperdiciaba la mayor parte de los datos.

Decisión. Entrenar B2 por lotes: `StandardScaler.partial_fit` con pesos $1/n$ y `SGDClassifier` con pérdida logística y penalización L2, leyendo los Parquet en grupos.
Por qué. Permite usar todo el entrenamiento sin cargarlo en memoria. Se convirtió C en $\alpha = 1/(C \cdot G)$, con G las partidas, para conservar el mismo objetivo ponderado de la regresión logística.
Descartado. Entrenar con una muestra: el baseline no habría visto los mismos datos que los demás modelos.

Bloqueo. Las pruebas de equivalencia detectaron que, con `average=True`, scikit-learn dejaba de actualizar el intercepto al encadenar `partial_fit`.
Impacto. B2 habría dado un modelo distinto del que dice describir.
Solución. Se promediaron los coeficientes a mano al terminar cada época. Una medición sintética de 2 millones de estados confirmó un pico de solo 279 MiB.

2026-10-19 • Primera búsqueda de B2 y lectura de la falta de convergencia

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Se compararon nueve valores de C, de 10^{-4} a 10^{4}, con cinco pliegues por partida en A, B y C. Desde $C = 0.1$ las métricas se aplanan. Ningún pliegue cumplió el criterio de convergencia (gradiente $\leq 10^{-3}$).

Decisión. No relajar la tolerancia para que el indicador dijera "convergió". Volver a entrenar solo las finalistas ($C = 0.1$ y $C = 1$) con hasta 600 épocas.
Por qué. "0 de 5" significa que el criterio no pudo certificar el mínimo, no que el modelo haya fallado. Cambiar el umbral solo para cambiar el indicador habría ocultado el problema.
Descartado. Bajar la tolerancia o reportar la búsqueda como concluyente.

2026-10-19 a 2026-10-23 • Reanudación y auditoría del costo

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Colab cortaba las sesiones a mitad de los entrenamientos. Se agregaron puntos de control por pliegue y por *trial*, y un orquestador que ejecuta B2, B3 y B4 en secuencia. Cuando B3 empezó a tardar horas sin avanzar, se auditó el código (`23-10-26_AuditoriaBaselinesYCoste.md`) y se encontraron cuatro problemas:

- Al reanudar, el muestreador de Optuna se recreaba con la misma semilla y **repetía configuraciones ya evaluadas**. Se reprodujo: el tercer *trial* repitió los parámetros del primero.
- Con 10 *trials* y 10 de arranque aleatorio, **la búsqueda bayesiana nunca llegaba a guiarse** por los resultados.
- Tres estudios por métrica multiplicaban el costo: 900 ajustes de B3 y 450 de B4, sin contar los finales.
- La base SQLite de Optuna vivía directamente en Google Drive, con escrituras cada minuto.

Decisión. Versión `efficient_v2`: cinco *trials* por estudio, tres de arranque aleatorio, espacio de búsqueda acotado (por ejemplo, bosques de 25 a 100 árboles con profundidad 8 o 12), SQLite en disco local con respaldos consistentes y reanudación que reencola el *trial* interrumpido.

Por qué. Elimina las configuraciones de costo extremo que hacían inviable el presupuesto, sin perder los resultados ya obtenidos. Las rutas nuevas no se mezclan con las anteriores.

Descartado. Seguir con el presupuesto original, que no terminaba en semanas.

2026-10-25 • Selección final de B2

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Tabla 3.2: B2: finalistas en validación (`25-10-26_B2SeleccionFinalC.md`).

Escenario	C	AUC	Log loss	Brier
A	**0.1**	**0.7607**	**0.5774**	**0.1975**
A	1.0	0.7607	0.5775	0.1975
B	**0.1**	**0.7281**	**0.6309**	**0.2140**
B	1.0	0.7278	0.6324	0.2143
C	**0.1**	**0.7603**	**0.5786**	**0.1977**
C	1.0	0.7602	0.5787	0.1977

Decisión. $C = 0.1$ en los tres escenarios, y cerrar B2 con el presupuesto invertido.

Por qué. Gana en las tres métricas en los tres escenarios. Los gradientes finales (0.0035 a 0.0072) no certifican el mínimo, y eso se reporta junto a las métricas.

Descartado. Seguir aumentando épocas sin límite, o usar prueba para resolver el empate.

2026-10-25 a 2026-10-26 • B4: una sola búsqueda, sin fuga entre escenarios

Responsables: Daniel y Adriel • Rama: `04MainAIModelTrainingAndDevelopment`

Daniel preparó un cuaderno independiente para B4. Adriel revisó su lógica antes de lanzar el entrenamiento completo y encontró un riesgo: buscar hiperparámetros por separado en cada escenario usaba partidas que eran validación o prueba en *otro* escenario.

Decisión. Una sola búsqueda (Optuna TPE, 20 *trials*, tres pliegues por partida) sobre las 11 285 partidas que son entrenamiento en A, B y C a la vez. Un solo estudio optimiza la pérdida logarítmica y registra AUC y Brier.

Por qué. Así ningún hiperparámetro vio partidas de validación, calibración o prueba de ningún escenario. Un estudio en lugar de tres divide el costo entre tres.

Descartado. Una búsqueda por escenario, que filtraba información entre escenarios.

Bloqueo. En el primer reentrenamiento final, con tope de 2000 árboles, los tres escenarios llegaron al tope: el modelo seguía mejorando.

Impacto. El número de árboles no lo decidía la parada temprana sino el tope.

Solución. Se subió el tope a 4000. La parada temprana se detuvo sola en 2888, 2370 y 2714 árboles, y las métricas cambiaron menos de 0.002. Se usó un servidor con 8 núcleos y 51 GiB de RAM.

Hito. B4 en validación: AUC 0.835 (A), 0.786 (B) y 0.836 (C).

Evidencia. Cuaderno `05_entrenamiento_B4_independiente_original.ipynb`, conservado como evidencia.

2026-10-28 • Selección de B3, definición de H2 y protocolo de LightGBM

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Decisión. Usar en B3 una sola configuración común, la de mayor macro-AUC de validación: el promedio simple del AUC de A, B y C.

Por qué. Se necesitaba una configuración compartida, y cada escenario debe pesar lo mismo. El macro-AUC solo elige; los resultados siguen reportándose por escenario.

Descartado. Elegir una configuración distinta por escenario, o promediar log loss y Brier en un puntaje arbitrario.

Tabla 3.3: B3: finalistas con los mismos hiperparámetros en los tres escenarios (AUC en validación).

Finalista	A	B	C	Macro-AUC
Árbol / log loss	0.7740	0.7343	0.7791	0.7624
Árbol / Brier	0.7767	0.7344	0.7810	0.7641
Bosque / Brier	0.8016	0.7621	0.8038	0.7892
Bosque / log loss	**0.8029**	**0.7609**	**0.8063**	**0.7901**

Decisión. Definir H2 como la comparación entre B4 con las 62 características y B4 restringido a cuatro entradas: turno, quién empezó y los identificadores de los dos Pokémon activos.

Por qué. La hipótesis pregunta si la señal temprana se explica solo por el orden de turno y la identidad del mazo. Los activos son la mejor aproximación a esa identidad disponible en cada estado sin usar datos del resto de la partida.

Descartado. Usar los mazos completos, que solo se conocen al terminar la partida.

Decisión. Cambiar B4 por LightGBM solo si, en cada escenario, el intervalo del 95 % de la diferencia de AUC queda entero por encima de cero y la pérdida logarítmica y el Brier no empeoran. Presupuesto igual al de B4.

Por qué. La regla se fijó antes de ejecutar, para que no pudiera ajustarse al resultado.

Descartado. Elegir el que saliera mejor sin regla previa.

Uso de IA: herramienta: Claude • para qué: proponer la regla de LightGBM (decisión delegada) • verificado: Daniel la documentó antes de ejecutar

2026-10-28 a 2026-10-29 • Auditoría de B0 a B3 y Kaggle como almacén único

Responsables: Daniel y Adriel • Rama: `04MainAIModelTrainingAndDevelopment`

Se auditó que B0 a B3 cumplieran el contrato de modelado del diccionario de datos (`29-10-26_AuditoriaBaselinesB0B3.md`): carga con filas utilizables, transformaciones ajustadas solo con entrenamiento, validación agrupada, roles de cada conjunto, 62 características sin identificadores y métricas por etapa. Todos cumplieron. También se unificó la evaluación en una sola función para todos los modelos.

Bloqueo. Los resultados vivían en Google Drive personal y en archivos `.zip` descargados a mano, y los `.joblib` no llegaban a GitHub por el `.gitignore`.

Impacto. Cada integrante veía una versión distinta de los resultados.

Solución. Adriel reemplazó Drive por el dataset privado de Kaggle como almacén único (`store.py`). Cada archivo publicado registra su SHA-256, el commit que lo produjo y la fecha, y cada publicación conserva todo lo anterior.

Decisión. Cuadernos sin interruptores: cada celda declara los archivos que produce y llama a `store.asegurar`. Si existen en Kaggle, los baja y verifica; si faltan, entrena y publica.

Por qué. Un interruptor olvidado en `True` puede reentrenar y sobrescribir horas de trabajo; uno en `False`, saltarse un paso. Sin interruptores, ejecutar la celda siempre hace lo correcto.

Descartado. Variables como `EJECUTAR = False` o `CONSTRUIR = True`.

2026-10-29 a 2026-10-30 • LightGBM, ablaciones, H2 y modelo final

Responsables: Adriel • Rama: `04MainAIModelTrainingAndDevelopment`

Hito. LightGBM no cumple la regla y se mantiene B4: AUC LightGBM/B4 de 0.8346/0.8346 (A), 0.7849/0.7863 (B) y 0.8352/0.8355 (C). Dos implementaciones independientes coinciden, así que el resultado no depende de la librería.

Evidencia. Carpeta `lightgbm/` en Kaggle.

Hito. Ablaciones: sin recursos y tablero el AUC cae de 0.05 a 0.06; sin progreso, como mucho 0.003; sin diferenciales no cae.

Evidencia. Carpeta `b4/ablaciones/`.

Decisión. No quitar los diferenciales de B4 aunque no aporten.

Por qué. Elegir características mirando validación sería una selección sobre el mismo conjunto que se usa para comparar.

Descartado. Simplificar el modelo con esas cifras.

Bloqueo. `HistGradientBoosting` admite como máximo 255 categorías por columna, y H2 necesita 359 cartas activas.

Impacto. Las salidas rápidas, agrupar o descartar las cartas poco frecuentes, perdían información de identidad, que es justo lo que H2 mide.

Solución. Adriel exigió conservar las 359 cartas. Cada identificador se repartió sin pérdida en bloques de hasta 255 categorías, ordenados por frecuencia. Esto no obligó a reentrenar B4.

Decisión. El modelo final para la Etapa 3 es B4 entrenado en A, expuesto en `ptcg_wp.predictor.PredictorVictoria`.

Por qué. A tiene la población más amplia y reciente; B y C sirven para medir generalización. El predictor recibe la observación del juego y produce las mismas características, en el mismo orden, que la tabla de entrenamiento.

Descartado. Elegir el modelo de B o C, o entrenar uno nuevo con todos los datos.

Hito. Escalera completa en validación (Tabla 3.4) y modelo exportado que reproduce las métricas de validación de B4 con diferencia máxima de 5.6×10^{-16}.

Evidencia. PR #9 y carpeta `modelo_final/` en Kaggle.

Tabla 3.4: AUC en validación de la escalera de modelos al cerrar la rama 04.

Escenario	B0	B1	B2	B3	B4
A	0.500	0.677	0.761	0.803	0.835
B	0.500	0.668	0.728	0.761	0.786
C	0.500	0.671	0.760	0.806	0.836

Uso de IA: herramienta: Claude • para qué: revisar la búsqueda de B4, implementar el almacén en Kaggle, la codificación por bloques de H2 y el predictor • verificado: Adriel probó cada cambio en Colab y rechazó las propuestas con interruptores, con dependencia de Drive o con pérdida de categorías

3.5. Rama 05: evaluación final

2026-11-02 • Diagnóstico y protocolo congelado antes de abrir prueba

Responsables: Sebastián • Rama: `05FinalModelChecks`

Sebastián reconstruyó el estado del proyecto (`02-11-26_DiagnosticoFinalModelChecks.md`) y encontró tres riesgos de método antes de escribir el protocolo:

- El orden sugerido del cuaderno dejaba SHAP y la revisión de fuga para después de prueba. Así no podían funcionar como barrera.
- Comparar directamente la caída del AUC con la del log loss favorecía a H3, porque están en escalas distintas.
- Comparar etapas con todas las partidas mezcla duración y progreso: las partidas cortas nunca llegan a la etapa final.

Decisión. Congelar el protocolo con un commit antes de abrir prueba (`02-11-26_ProtocoloEvaluacionFinal.md`), con este orden: verificar la carga de modelos, SHAP y revisión de fuga, decidir la calibración solo con calibración, curvas de aprendizaje y, solo entonces, prueba.

Por qué. Si una regla se escribe después de ver el resultado, puede acomodarse a él. La celda de prueba se niega a correr si falta alguna barrera.

Descartado. Evaluar prueba primero y decidir después.

Decisión. Decidir H3 con la descomposición de la pérdida en calibración (MCB) y ordenamiento (DSC), sobre las mismas 1 405 partidas de prueba de A y B.

Por qué. Deja ambas pérdidas en las mismas unidades. Usar las mismas partidas aísla el efecto del entrenamiento del de la composición del conjunto.

Descartado. Comparar porcentajes de caída de AUC y de log loss.

Decisión. Controlar la duración en H1 con una cohorte: solo las partidas que llegan a las tres etapas.

Por qué. Si el AUC sube de inicio a final dentro de las mismas partidas, el aumento no viene de que las partidas largas sean más fáciles.

Descartado. Comparar etapas con todas las partidas.

Decisión. Revisión de fuga con SHAP: aprueba si ninguna columna prohibida participa, si ninguna característica concentra más del 50 % de la importancia y si ninguna característica sola alcanza un AUC de 0.85.

Por qué. Una variable que domina el modelo suele ser la firma de una fuga. Los umbrales se fijaron antes de mirar.

Descartado. Revisar las gráficas SHAP a ojo.

Uso de IA: herramienta: Claude Code • para qué: diagnóstico, protocolo, implementación del cuaderno 06 y sus pruebas sintéticas • verificado: Sebastián revisó y aprobó el protocolo; la herramienta no tuvo acceso a los datos

2026-11-02 a 2026-11-03 • Ejecución del cuaderno 06

Responsables: Adriel • Rama: `05FinalModelChecks`

Adriel ejecutó el cuaderno de arriba abajo en Colab Pro, sin que nadie más publicara en Kaggle al mismo tiempo. Antes de prueba pasaron todas las barreras: 109 pruebas automáticas, reproducción exacta de las métricas de validación (diferencias $\leq 4.4 \times 10^{-16}$ en log loss y Brier, y $\leq 4.8 \times 10^{-7}$ en AUC) y revisión de fuga aprobada (importancia máxima 0.113; mejor característica sola con AUC 0.690).

Bloqueo. La publicación del análisis de errores se marcó como fallida aunque había funcionado: los archivos `errores_*.csv` contienen la palabra “error”, y `store.py` la confundía con un mensaje de Kaggle.

Impacto. La celda se detuvo después de abrir prueba.

Solución. Se corrigió la detección para leer solo los mensajes de error reales y se agregaron pruebas. Al volver a ejecutar, lo publicado se bajó sin recalcular: prueba no se volvió a predecir.

Bloqueo. Justo después de publicar, el índice del dataset aún no mostraba la versión nueva a otros servidores.

Impacto. Una celda podía creer que un archivo faltaba.

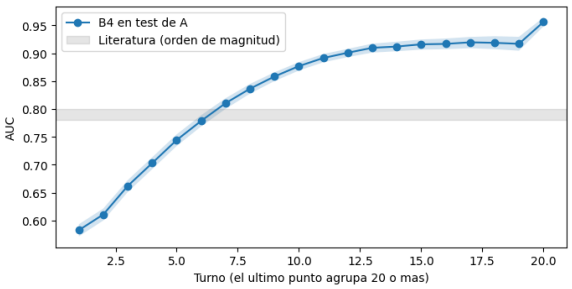
Solución. El almacén guarda una marca de publicación y prefiere la copia más reciente; si Kaggle no responde, se espera unos minutos y se reintenta.

2026-11-04 a 2026-11-05 • Resultados y veredicto

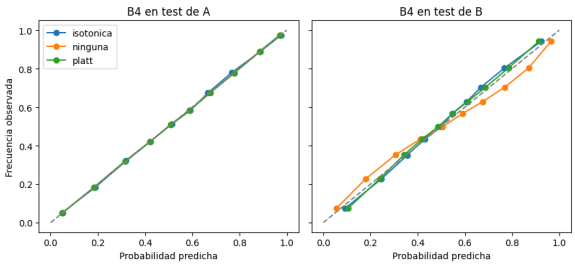
Responsables: Sebastián y Adriel • Rama: `05FinalModelChecks`

Tabla 3.5: AUC de B4 en prueba (intervalo del 95 %) y veredicto de las hipótesis.

	A	B	C	Lectura
AUC global	0.833	0.789	0.787	Supera a B0–B3 con intervalos que excluyen el cero
AUC inicio / final	0.684 / 0.937	0.630 / 0.915	0.623 / 0.919	La predicción mejora con la partida
H1	Sostenida	Sostenida	Sostenida	Cohorte: +0.32 de inicio a final
H2	Refutada	Sostenida	Sostenida	El estado aporta cuando cambia la población
H3		Refutada		El ordenamiento se degrada más que la calibración



(a) AUC de B4 por turno (A).



(b) Diagramas de fiabilidad de B4.

Figura 3.2: Dos resultados de la evaluación final, tomados del cuaderno 06.

Decisión. Usar el modelo final sin calibrador.

Por qué. En A, B4 ya está calibrado (ECE 0.002) y la isotónica lo empeora un poco. En B la isotónica baja el ECE de 0.037 a 0.018, pero la misma estrategia empeoró a B3: corregir bajo deriva no es fiable en general. La elección se hizo con calibración, antes de abrir prueba.

Descartado. Calibrar siempre por defecto.

Decisión. No agregar una comprobación extra de procedencia de los archivos de prueba.

Por qué. El cuaderno 06 se versiona con sus salidas, y el índice del dataset ya registra el SHA-256, el commit y la fecha de cada archivo. La comprobación no añadía información nueva.

Descartado. Una celda adicional que repitiera esa verificación.

Hito. Evaluación final cerrada: 51 archivos de resultados en `evaluacion_final/`, verificados con SHA-256.

Evidencia. PR #10, `04-11-26_ResultadosEvaluacionFinal.md` y cuaderno `06` con salidas.

3.6. Rama 06: artículo y cierre

2026-11-05 a 2026-11-06 • Corrección del artículo y documentación final

Responsables: Adriel • Rama: `06PendientesDelArticulo`

Se corrigieron en el artículo los datos que la Etapa 2 cambió: el corpus final (55 196 partidas), el conteo de premios, los días del escenario B, la definición del escenario C por arquetipos, la cobertura de cartas medida, el sesgo de la posición 0 y la URL real del conjunto de datos. Después se escribieron las secciones de resultados, discusión y conclusiones, y se actualizó la documentación del repositorio.

Decisión. Declarar en el artículo las desviaciones respecto del diseño: una sola semilla, ablaciones sobre tres familias, sin coeficientes de B2 ni importancia por permutación, y escenario C por arquetipos.

Por qué. Un cambio no declarado haría que el lector crea que se ejecutó el diseño original.

Descartado. Omitirlas o reescribir la metodología como si siempre hubiera sido así.

Hito. Artículo con metodología definitiva, resultados, discusión y conclusiones.

Evidencia. Commits de la rama `06PendientesDelArticulo`.

3.7. Cómo protegimos los datos

La Tabla 3.6 reúne las medidas contra la fuga de información. Cada una responde a una vía concreta por la que el modelo podría ver algo que no debería.

Tabla 3.6: Vías de fuga y cómo se cerró cada una.

Vía de fuga	**Medida**	**Comprobación**
Estados de una partida a ambos lados de una partición	Toda partición, pliegue, vigilancia e intervalo se agrupa por partida	Pruebas de particiones
Información oculta (mano rival, premios)	Solo se lee la vista del jugador que actúa; los campos ocultos nunca entran	Pruebas que llenan lo oculto y exigen X idéntico
Resultado, recompensas y pasos futuros	Columnas prohibidas por código	Pruebas de fuga
Estados con el desenlace a la vista	Filtro `usable` (sin preparación ni estados resueltos)	Prueba de carga
Futuro en el escenario B	Días completos en orden; no se mueven partidas futuras a entrenamiento	Prueba de cronología
Hiperparámetros que ven otros escenarios	Búsqueda de B4 solo con las 11 285 partidas de entrenamiento comunes	Prueba de la búsqueda
Preprocesamiento ajustado con todos los datos	Escalado y cortes de etapa ajustados solo con entrenamiento	Auditoría de B0–B3
Decisiones tomadas con prueba	Protocolo congelado; prueba se abre una vez, después de todas las barreras	Celda de prueba con barreras
Una variable que filtra el resultado	Revisión SHAP con umbrales previos	Aprobada antes de prueba

3.8. Cómo aseguramos que todo funcionara

- **Pruebas automáticas.** Pasaron de 24 al cerrar la rama de datos a 109 al ejecutar la evaluación final. Cubren fuga, particiones, evaluación, almacén, entrenamiento y predictor, con datos sintéticos.
- **Valor esperado en cada celda.** Cada celda anota lo que debe mostrar (por ejemplo, "55 196 partidas"). Si la salida no coincidía, no se hacía commit.
- **Reproducción antes de avanzar.** El modelo exportado y el cuaderno 06 debían reproducir las métricas ya publicadas antes de usarse.
- **Trazabilidad de cada archivo.** Cada resultado en Kaggle guarda su SHA-256, el commit que lo produjo y la fecha.
- **Reanudación segura.** Puntos de control por pliegue, *trial* y escenario, con manifiestos que impiden mezclar configuraciones distintas.
- **Una sola persona publica.** Al ejecutar algo nuevo, se avisaba al grupo para que nadie más publicara al mismo tiempo.

3.9. Registro de experimentos

Semilla 2026 en todas las corridas, salvo B3 (semilla efectiva 2030). Las cifras de AUC van en el orden A / B / C.

Tabla 3.7: Corridas de la escalera de modelos (B0 a B4).

Corrida	**Configuración**	**Fecha**	**Resultado**
B0 y B1	Prevalencia; heurística de recursos con pendiente e intercepto ajustados en entrenamiento	13/10	AUC de B1 en validación 0.668–0.677
B2, búsqueda	9 valores de C, 5 pliegues, todo el entrenamiento	19/10	Métricas planas desde $C = 0.1$; 0 de 5 pliegues convergen
B2, finalistas	$C \in \{0.1, 1\}$, hasta 600 épocas	25/10	$C = 0.1$; AUC 0.761 / 0.728 / 0.760
B3, `efficient_v2`	Árbol y bosque, 5 *trials* por estudio	23–28/10	Bosque común por macro-AUC; AUC 0.803 / 0.761 / 0.806
B4, búsqueda	Optuna TPE, 20 *trials*, 3 pliegues, 11 285 partidas	26/10	Configuración del *trial* 20
B4, tope 2000	Reentrenamiento final	26/10	Los tres escenarios llegaron al tope
B4, tope 4000	Reentrenamiento final	26/10	2888 / 2370 / 2714 árboles; AUC 0.835 / 0.786 / 0.836

Tabla 3.8: Corridas de experimentos y evaluación final.

Corrida	**Configuración**	**Fecha**	**Resultado**
LightGBM	Mismo presupuesto que B4	29–30/10	No cumple la regla; se mantiene B4
Ablaciones	B4 sin cada familia	29–30/10	Recursos y tablero es la familia esencial
H2	B4 restringido a 4 entradas, 359 cartas sin pérdida	30/10	AUC 0.733 / 0.614 / 0.743 en validación
Curvas de aprendizaje	25, 50, 75 y 100 % del entrenamiento	03/11	El último 25 % suma ≤ 0.004
Evaluación final	Una sola vez en prueba	03/11	AUC 0.833 / 0.789 / 0.787

3.10. Decisiones consolidadas

Tabla 3.9: Decisiones de la Etapa 2: datos y baselines (11/10 a 25/10).

Fecha	Decisión	Por qué	Artículo
11/10	Excluir preparación y estados resueltos	Tienen el desenlace a la vista	Metodología
11/10	Escenario B por días completos, sin el 17 de junio	Evita mezclar presente y futuro	Metodología
11/10	Escenario C por 33 arquetipos	El corpus no trae mazos completos	Metodología
11/10	Cobertura de cartas en A y C, no en B	En B sería fuga temporal	Metodología
12/10	Peso $1/n$ por partida	Una partida larga no debe pesar más	Metodología
12/10	B1 con pesos fijos	Responde si hace falta aprendizaje	Resultados
18/10	B2 por lotes con todo el entrenamiento	Cabe en Colab sin muestrear	Resultados
19/10	No relajar la tolerancia de B2	No ocultar la falta de convergencia	Resultados
23/10	Búsquedas `efficient_v2`	El presupuesto original no terminaba	Resultados
25/10	$C = 0.1$ en B2	Mejor en las tres métricas	Resultados

Tabla 3.10: Decisiones de la Etapa 2: B4, experimentos y evaluación final (26/10 a 06/11).

Fecha	Decisión	Por qué	Artículo
26/10	Una sola búsqueda de B4 en partidas comunes	Sin fuga entre escenarios	Resultados
26/10	Tope de 4000 árboles	El tope de 2000 se alcanzaba	Resultados
28/10	Macro-AUC para B3	Una configuración común, escenarios con igual peso	Resultados
28/10	Regla de LightGBM previa	No acomodar la regla al resultado	Resultados
29/10	Kaggle como almacén único y cuadernos sin interruptores	Una sola versión de los resultados, con trazabilidad	Resultados
30/10	H2 con las 359 cartas sin pérdida	H2 mide justo la identidad	Resultados
30/10	Modelo final: B4 de A	Población más amplia	Conclusiones
02/11	Protocolo congelado y barreras antes de prueba	Prueba se usa una vez	Resultados
02/11	H3 con descomposición MCB/DSC	Escalas comparables	Resultados
04/11	Modelo final sin calibrador	Decidido con calibración, no con prueba	Resultados
06/11	Declarar las desviaciones del diseño	Transparencia	Resultados

3.11. Retrospectiva

Tabla 3.11: Retrospectiva de la Etapa 2.

Pregunta	Respuesta
¿Qué funcionó?	Cerrar cada bloque antes del siguiente. Escribir las reglas antes de ejecutar (acuerdos, protocolo de LightGBM, protocolo final). Probar la fuga con pruebas que intentan provocarla. Kaggle como almacén único con trazabilidad por archivo.
¿Qué nos bloqueó?	La RAM y los cortes de sesión de Colab, y el costo de las búsquedas de B3 y B4 con el presupuesto original. El almacén en Drive, que dejaba versiones distintas en cada integrante.
¿Qué estimamos mal?	El tiempo de entrenamiento: B2 y B3 tomaron unos cuatro y tres días de calendario por las reanudaciones. También el tope de árboles de B4, que hubo que duplicar.
¿Qué cambia para la Etapa 3?	Partir del predictor ya exportado y probado, mantener el almacén en Kaggle y las celdas sin interruptores, y fijar por escrito cómo se verificará el consumo del modelo antes de construir el agente.

4. Etapa 3: agente, validación, ética y artículo final

Objetivo de la etapa. Convertir el modelo final de la Etapa 2 en un componente que una persona pueda usar a través de un agente, validar ese agente con un protocolo escrito antes de ejecutar, declarar la ética de su despliegue y cerrar el artículo. El modelo no se reentrena ni se recalibra. Peso: 25 %.

4.1. Resumen de ramas

Tabla 4.1: Ramas de la Etapa 3 y su resultado.

Rama	Fechas	Responsables	Resultado
`07MultiAgentIntegration`	07/11	Adriel	Herramientas del modelo, cuatro agentes, registro, servidor MCP, orquestador y subagentes de Claude Code (PR #13)
`08AgentValidationAndSimulator`	08/11	Adriel; validación humana de los tres	Validación N1 y N2, análisis de fragilidad y simulador gráfico (PR #14)
`09EthicsPaperAndDocumentation`	08/11	Adriel; revisión de Daniel y Sebastián	Ética del despliegue, artículo final, documentación y uso de IA
`10DefenseAndShowcase`	Semana 16	Equipo	Pendiente: defensa, video y entrega

4.2. Lo que recibió de la Etapa 2

Tabla 4.2: Piezas que la Etapa 2 dejó listas para el agente.

Pieza	Detalle
Modelo final	B4 entrenado en el escenario A, sin calibrador. En Kaggle, `modelo_final/b4_victoria.joblib` con su ficha (origen, SHA-256, versión de scikit-learn y métricas)
Interfaz	`ptcg_wp.predictor.PredictorVictoria`: recibe la observación del jugador que actúa y devuelve la probabilidad de que gane. En preparación y en estados ya resueltos devuelve `None`
Garantía de consistencia	`tests/test_predictor.py` comprueba que la observación produce las mismas características, en el mismo orden, que la tabla de entrenamiento
Explicaciones	SHAP mostró que el modelo decide por premios, descartes y mazo; las explicaciones del agente se dan en esos términos

También recibió tres condiciones de uso: desconfiar de las probabilidades extremas al inicio de la partida, comprobar que el agente no sobrevalore dejar al rival sin mazo y no confiar en la calibración con datos de otro periodo sin volver a medirla. Las tres quedaron como advertencias del agente o como límites declarados.

4.3. Cómo se construyó, parte por parte

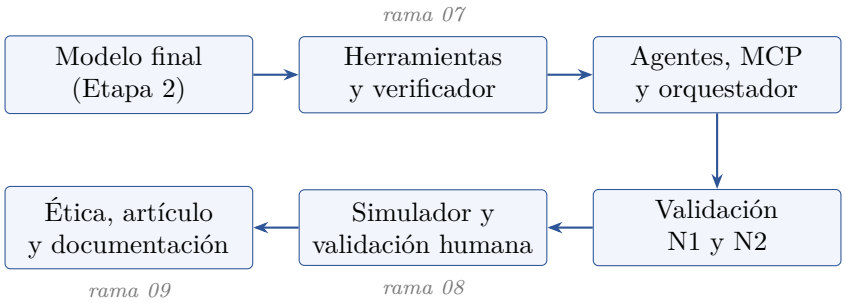


Figura 4.1: Orden de construcción de la Etapa 3. Cada pieza se probó antes de empezar la siguiente.

4.4. Rama 07: integración del agente

2026-11-07 • Herramientas, agentes y servidor MCP

Responsables: Adriel • Rama: `07MultiAgentIntegration`

Se escribió primero la arquitectura (`07-11-26_ArquitecturaDelAgente.md`) y después el código en `src/ptcg_agente/`, separado del código de entrenamiento. Hay seis herramientas del modelo (estimar, explicar, evaluar una partida, comparar estados, revisar el dominio y describir el modelo), cuatro agentes y un registro JSONL de cada

invocación. El agente se usa desde Claude Code o Claude Desktop, con el servidor MCP `ptcg-victoria`, o desde un orquestador en Python.

Decisión. Usar la membresía de Claude del equipo, sin clave de API: Claude Code y Claude Desktop con un servidor MCP, y Claude Code sin interfaz (`claude -p`) para el planificador y el redactor del orquestador.

Por qué. Una clave de API sería otra credencial que proteger y otro costo. Con la membresía, cada integrante usa su propia cuenta.

Descartado. Una clave de API de Anthropic o un modelo local más débil.

Decisión. El planificador y el redactor son dos llamadas separadas a Claude, sin herramientas; el ejecutor y el verificador son código.

Por qué. Así el LLM nunca calcula una probabilidad: solo decide qué herramientas llamar y redacta con resultados ya aprobados. El verificador revisa que cada cifra de la respuesta salga de una herramienta, que aparezcan las advertencias obligatorias y que no se prometa certeza.

Descartado. Un solo agente con todas las herramientas, que mezcla decidir, calcular y redactar en la misma llamada.

Hito. El agente da la misma probabilidad que el modelo final en 642 estados de 4 partidas reales (diferencia máxima 0.0).

Evidencia. Parte 3 del cuaderno `07_agente`; sesión de Claude Code en `08-11-26_RegistroSesionClaudeCode.jsonl` y `.png`, donde el verificador rechazó un primer borrador y aprobó el corregido.

Uso de IA: herramienta: Claude • para qué: proponer la arquitectura e implementar las herramientas, los agentes, el servidor MCP y sus pruebas • verificado: Adriel ejecutó el cuaderno 07, instaló el agente en su PC y revisó cada respuesta guardada como evidencia

4.5. Rama 08: validación y simulador

2026-11-08 • Protocolo y validación en dos niveles

Responsables: Adriel • Rama: `08AgentValidationAndSimulator`

El protocolo (`08-11-26_ProtocoloValidacionAgente.md`) fijó, antes de ejecutar, los escenarios, el comportamiento esperado y el criterio de aprobación de cada caso. La validación tiene dos niveles: N1 prueba las capas deterministas sin LLM (cuaderno 09, en Colab) y N2 prueba el agente completo con Claude (cuaderno 10, en la PC).

Bloqueo. Kaggle respondía 429 (demasiadas solicitudes) al bajar en paralelo las 166 repeticiones de los casos.

Impacto. La parte 1 del cuaderno 09 fallaba a mitad de la descarga.

Solución. Descargas una por una, con espera y reintento, en una carpeta que conserva lo ya bajado.

Bloqueo. Kaggle respondía 404 al bajar `partidas.json.gz`.

Impacto. El cuaderno 10 no encontraba las partidas de validación.

Solución. Guardar `partidas.json` sin comprimir y borrar el archivo anterior del dataset.

Bloqueo. La herramienta `kaggle` 2.0.2 de Colab respondía 400 al crear una versión, sin decir el motivo.

Impacto. No se podía publicar el catálogo de cartas del simulador.

Solución. Fijar `kaggle>=2.2.5` en `requirements.txt` y, si vuelve a ocurrir, repetir con la API de Python para mostrar el motivo sin imprimir el token.

La primera corrida de N1 dio 32 de 35 casos aprobados, 2 fallidos y 1 de registro. Los dos fallos eran de presentación, no del modelo:

- **L-09.** La partida más larga (1398 pasos) tardaba 56.6 s porque se llamaba al modelo una vez por paso. Ahora se predice toda la partida en una llamada: 0.9 s, con las mismas probabilidades.
- **L-10.** Sin Pokémon activo, la explicación mostraba el valor interno -1 como si fueran puntos de vida. Ahora dice "no hay Pokémon activo".

Decisión. Reportar la corrida original junto a la corregida, sin cambiar los criterios del protocolo.

Por qué. Mostrar solo la corrida corregida ocultaría que la validación sí encontró defectos reales.

Descartado. Ajustar los umbrales de L-09 o borrar la primera corrida.

Hito. N1 corregido: 34 de 35 casos aprobados y 1 de registro. N2: 31 de 31 consultas aprobadas, herramienta correcta en 18 de 18, cifras trazables en 31 de 31, 28 de 31 aprobadas en la primera ronda del redactor, 0 respuestas en la plantilla, latencia p95 de 31 s y costo equivalente de 1.72 USD.

Evidencia. `08-11-26_ResultadosValidacionAgente.md`, `08-11-26_ValidacionN1Corregida.csv` y `08-11-26_ValidacionN2.csv`.

El hallazgo más importante fue de fragilidad: 50 de 60 errores muy seguros de medio y final de la partida llegan sin advertencia, y el agente los presenta como fiables. El

verificador detecta cifras inventadas y advertencias omitidas, pero no puede saber cuándo el modelo se equivoca con seguridad.

2026-11-08 • Simulador gráfico y validación humana

Responsables: Adriel; los tres en la validación humana • Rama: `08AgentValidationAndSimulator`

El simulador (`python -m ptcg_agente.simulador`) es una página web local que muestra el tablero de los dos jugadores, deja editar el estado y recalcula la probabilidad, las contribuciones SHAP y las advertencias en cada cambio. Usa solo la biblioteca estándar de Python y JavaScript sin librerías. El agente lo abre con la herramienta `abrir_simulador`.

Decisión. Jugadores fijos en pantalla (el 1 arriba, el 0 abajo) y la cifra grande siempre del jugador 0.

Por qué. En la primera versión la cifra era del jugador que actuaba, así que cambiaba de dueño al avanzar la partida y confundía al usuario.

Descartado. Mostrar la probabilidad del jugador que actúa, que es la salida directa del modelo.

Decisión. Agregar un "estado nuevo" vacío (turno 0, sin cartas ni Pokémon) y un catálogo con los PV máximos de las 362 cartas vistas como activo.

Por qué. Permite armar un estado desde cero, sin depender de una partida, y elegir cualquier carta con sus PV reales.

Descartado. Solo editar estados de partidas reales.

Decisión. En la validación humana, 5 pruebas distintas por integrante (15 en total) con la dirección esperada fijada antes.

Por qué. Una primera versión repetía las mismas pruebas para los tres y no añadía información.

Descartado. Tres sesiones idénticas.

Hito. El simulador funcionó en las tres computadoras y 10 de 15 pruebas tuvieron la dirección esperada: todas las de premios, mazo, PV y banca. Las condiciones especiales casi no movieron la probabilidad, y más energías en el activo la bajaron (asociación, no causa).

Evidencia. `08-11-26_ValidacionHumanaSimulador.md` y las capturas `08-11-26_Simulador*.png`.

Uso de IA: herramienta: Claude • para qué: implementar la batería, los cuadernos 09 y 10 y el simulador, y diagnosticar los errores de Kaggle • verificado: Adriel ejecutó N1 y N2 y revisó a mano Q-01, Q-02 y Q-07; cada integrante hizo sus propias pruebas con el simulador

4.6. Rama 09: ética, artículo y documentación

2026-11-08 • Ética del despliegue y artículo final

Responsables: Adriel; revisión de Daniel y Sebastián • Rama: `09EthicsPaperAndDocumentation`

La ética (`08-11-26_EticaDelDespliegue.md`) declara 8 riesgos con su gravedad y probabilidad, los datos sensibles, una mitigación con su archivo y su prueba para cada riesgo, 9 límites y el uso aceptable. El artículo sumó la sección del agente (arquitectura, protocolo, resultados y fragilidad), con la consulta real en Claude Code y el simulador como figuras; la ética se reescribió con foco en el despliegue; y la discusión y las conclusiones incluyen el agente y el objetivo 7.

Decisión. No ofrecer el sistema como servicio público: todo corre en la computadora del usuario.

Por qué. El sistema no puede impedir por sí mismo el uso en partidas en curso o en apuestas; sin un servicio público, ese uso es menos fácil y el uso aceptable queda declarado.

Descartado. Publicar el simulador en la web.

Decisión. Declarar como límite, y no ocultar, que la mayoría de los errores muy seguros no traen advertencia.

Por qué. Es el riesgo más grave (confianza excesiva) y el lector debe conocerlo. La mejora propuesta, mostrar con qué frecuencia fallan las predicciones muy seguras, queda como trabajo futuro.

Descartado. Agregar una advertencia nueva sin validarla con un protocolo.

Hito. Artículo final de 29 páginas que compila sin errores, con los siete objetivos cumplidos; documentación del repositorio actualizada para ejecutar el modelo, el agente y el simulador.

Evidencia. Commits de la rama `09EthicsPaperAndDocumentation`.

Daniel y Sebastián revisaron, con apoyo de Claude, que el desarrollo de la etapa siguiera el plan y las reglas del equipo y que todo funcionara bien según su propia evaluación humana, además de hacer sus pruebas de la validación humana (`01UsoDeIA/Etapa3UsoDeIA.md`).

4.7. Cómo aseguramos que el agente fuera confiable

- **Mismo modelo, comprobado.** El agente solo arranca si el modelo y sus archivos coinciden por SHA-256 con los publicados, y con la misma versión de scikit-learn.

- **Ninguna cifra inventada.** Toda probabilidad sale de una herramienta y queda en el registro; el verificador rechaza las respuestas con cifras que no salieron de una herramienta.
- **Sin información oculta.** Ni el modelo ni el agente usan la mano del rival ni el contenido de los premios; una prueba lo comprueba.
- **Pruebas automáticas.** Pasaron de 109 al cerrar la Etapa 2 a 145, con las del agente, el orquestador (con un LLM de prueba), el servidor MCP, la validación y el simulador. Corren como segunda celda de los cuadernos 07 a 10.
- **Protocolo antes de ejecutar.** Los criterios de aprobación se fijaron por escrito y no cambiaron al ver los resultados.
- **Evaluación humana.** Cada integrante probó el simulador en su computadora y revisó el funcionamiento por su cuenta.

4.8. Retrospectiva

Tabla 4.3: Retrospectiva de la Etapa 3.

Pregunta	Respuesta
¿Qué funcionó?	Separar decidir y redactar (LLM) de calcular y verificar (código). Partir de un predictor ya probado. Escribir el protocolo antes de ejecutar. Mantener Kaggle como almacén único también para los archivos del agente.
¿Qué nos bloqueó?	Los límites y cambios de Kaggle (429, archivos `.gz` y el error 400 de la herramienta vieja) y la espera mientras Kaggle procesa cada versión.
¿Qué estimamos mal?	El costo de evaluar una partida paso a paso (L-09) y la claridad del simulador: la primera versión, con la cifra del jugador que actúa, confundía.
¿Qué queda abierto?	Advertir cuándo una predicción muy segura suele fallar, que el modelo use mejor las condiciones especiales y la defensa con el video demostrativo (rama 10).

4.9. Trabajo pendiente heredado

- Repetir los experimentos principales con varias semillas.
- Agregar una característica con los premios que entrega cada Pokémon activo, que hoy el modelo aproxima con sus puntos de vida máximos.
- Entrenar la variante de 64 características (estado completo más identificadores de carta).

A. Reuniones y sesiones de trabajo

A.1. Índice

Las reuniones de la Etapa 1 se hicieron por llamada en Discord y quedaron registradas con capturas. Desde la Etapa 2 la coordinación pasó a sesiones de trabajo por rama y a

acuerdos escritos en `docx/03Evidencia/`, que funcionan como actas. La duración de las llamadas es la que muestra el contador de la captura, así que es una cota inferior.

Tabla A.1: Reuniones y sesiones registradas.

Fecha	Duración	Participantes	Tema principal
21/08	20 min o más	Adriel, Daniel, Sebastián	Primera reunión; conjuntos de datos en Kaggle
26/08	15 min o más	Adriel, Sebastián	Confirmación del tema
30/08	13 min o más	Adriel, Daniel, Sebastián	Cierre de la confirmación y reparto de ejes
06/09	1 h 38 min o más	Adriel, Daniel, Sebastián	Revisión final del artículo de la Etapa 1
12/10	Acta escrita	Daniel	Acuerdos técnicos de modelado
28/10	Acta escrita	Daniel	Selección de B3, definición de H2 y protocolo de LightGBM
02/11	Acta escrita	Sebastián	Protocolo de evaluación final

A.2. Actas

2026-08-21 • Primera reunión del equipo

Responsables: Adriel, Daniel, Sebastián • Rama: --

Temas.

- Revisión de conjuntos de datos públicos en Kaggle.
- Organización del repositorio.

Acuerdos.

- Adriel crea el repositorio con las reglas de ramas y commits (cumplido el mismo día, PR #1).
- El equipo considera las repeticiones del PTCG AI Battle Challenge como corpus candidato.

2026-08-26 • Confirmación del tema

Responsables: Adriel, Sebastián • Rama: --

Acuerdos.

- Track A: probabilidad de victoria en el Pokémon TCG desde el estado de la partida.
- Se deja la dirección de aprendizaje por refuerzo explorada antes.

2026-08-30 • Cierre de la confirmación del *track*

Responsables: Adriel, Daniel, Sebastián • Rama: --

Acuerdos.

- Base de investigación de la Etapa 1 cerrada (PR #2).
- Revisión de literatura repartida por ejes: Daniel el A, Adriel el B y Sebastián el C.

2026-09-06 • Revisión final del artículo

Responsables: Adriel, Daniel, Sebastián • Rama: `--`

Temas.

- Lectura completa del artículo antes de la entrega.
- Ajustes de formato, de la tabla comparativa y de la metodología.

Acuerdos.

- Integrar `02PaperAndDelivery` y `Stage1` a `master` (PR #4 y #5) y entregar.

2026-10-12 • Acuerdos técnicos de modelado

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Acta: `12-10-26_AcuerdosTecnicosModelado.md`. Fijó la fórmula de B1, la validación cruzada agrupada, los pesos por partida, la búsqueda de B2 en cuadrícula, las búsquedas de B3 y B4 con Optuna y la parada temprana por partidas. Dejó abiertas las semillas múltiples y el tipo de intervalo, que se cerraron en el protocolo final (una semilla; percentil del 95 % con 1000 remuestreos).

2026-10-28 • B3, H2 y LightGBM

Responsables: Daniel • Rama: `04MainAIModelTrainingAndDevelopment`

Actas: `28-10-26_SeleccionB3yDefinicionH2.md` y `28-10-26_ProtocoloLightGBM.md`. Fijaron el macro-AUC para elegir B3, la definición de las cuatro entradas de H2 y la regla para reemplazar B4 por LightGBM, todo antes de ejecutar.

2026-11-02 • Protocolo de evaluación final

Responsables: Sebastián • Rama: `05FinalModelChecks`

Acta: `02-11-26_ProtocoloEvaluacionFinal.md`. Fijó el orden de las barreras, las reglas de decisión de H1, H2 y H3, la calibración decidida con `calib` y los umbrales de la revisión de fuga. Se congeló con un commit antes de abrir prueba.